

Complete C# + .NET Learning Order

Tiered Roadmap for a Fullstack .NET Developer (Backend Focus, Angular Frontend)

This is a complete, dependency-ordered topic list for learning C# and .NET from zero to professional capability, organized into three tiers based on real job relevance for a Fullstack .NET Developer role. Tier 1 topics should be studied deeply and practiced through projects. Tier 2 topics should be understood conceptually and studied in depth only when a specific project or job need arises. Tier 3 topics require no study time — just awareness that they exist.

TIER 1 — MUST MASTER

Build deep fluency here through daily practice and projects. This is the 20% that delivers 80% of real-world capability.

0. Foundations Before Code

- 0.1. What is HTTP — request/response cycle, statelessness
- 0.2. HTTP verbs (GET, POST, PUT, PATCH, DELETE)
- 0.3. HTTP status codes (2xx, 3xx, 4xx, 5xx)
- 0.4. What is REST — resources, endpoints, conventions
- 0.5. What is a relational database — tables, rows, columns
- 0.6. Primary keys and foreign keys
- 0.7. Basic SQL — SELECT, INSERT, UPDATE, DELETE, WHERE, JOIN basics
- 0.8. What is an API — client/server model, JSON as data format

1. C# Language Fundamentals

- 1.1. .NET ecosystem overview (CLR, BCL, SDK)
- 1.2. Environment setup, project/solution structure
- 1.3. Main method, top-level statements
- 1.4. Variables, data types, value vs reference types
- 1.5. Type conversion (implicit, explicit, Parse/TryParse)
- 1.6. Operators (arithmetic, logical, comparison, null-coalescing ?? and ??=)
- 1.7. Control flow — if/else, switch statements and expressions
- 1.8. Loops — for, foreach, while, do-while
- 1.9. Arrays (single, multi-dimensional, jagged)
- 1.10. Strings — manipulation, interpolation, StringBuilder
- 1.11. Methods — parameters, return types, overloading, ref/out/in
- 1.12. Optional and named parameters
- 1.13. Exception handling — try/catch/finally, custom exceptions
- 1.14. Nullable types and nullable reference types
- 1.15. Enums
- 1.16. Structs vs classes
- 1.17. Comments and XML documentation comments
- 1.18. Constants — const vs readonly
- 1.19. Scope and variable lifetime

- 1.20. var and implicit typing
- 1.21. Pattern matching — is, switch expressions, property patterns
- 1.22. Tuples and deconstruction
- 1.23. Namespaces and using directives

2. Object-Oriented Programming in C#

- 2.1. Classes and objects
- 2.2. Constructors (default, parameterized, static, primary constructors)
- 2.3. Properties (get/set, auto-properties, init-only)
- 2.4. Access modifiers (public, private, protected, internal, etc.)
- 2.5. Encapsulation
- 2.6. Inheritance
- 2.7. Polymorphism (virtual, override)
- 2.8. Abstract classes
- 2.9. Interfaces (including default interface methods)
- 2.10. Sealed classes/methods
- 2.11. Static classes and members
- 2.12. Partial classes
- 2.13. Records (record class, record struct, with expressions)
- 2.14. Object initializers
- 2.15. this keyword
- 2.16. Method hiding vs overriding (new vs override)
- 2.17. Object equality — Equals, GetHashCode, == overloading

3. Generics

- 3.1. Generic classes and methods
- 3.2. Generic constraints (where)

4. Collections

- 4.1. List<T>
- 4.2. Dictionary<TKey, TValue>
- 4.3. HashSet<T>, SortedSet<T>
- 4.4. Queue<T>, Stack<T>
- 4.5. IEnumerable<T>, ICollection<T>, IList<T> interfaces
- 4.6. IComparable / IComparer for custom sorting

5. LINQ (Language Integrated Query)

- 5.1. Query syntax vs method syntax
- 5.2. Filtering (Where), projection (Select)
- 5.3. Ordering (OrderBy, ThenBy)
- 5.4. Grouping (GroupBy)
- 5.5. Aggregation (Sum, Count, Average, Min, Max, Aggregate)
- 5.6. Joins (Join, GroupJoin)
- 5.7. Quantifiers (Any, All, Contains)
- 5.8. Element operators (First, FirstOrDefault, Single, ElementAt)

- 5.9. Set operations (Distinct, Union, Intersect, Except)
- 5.10. Deferred vs immediate execution
- 5.11. IQueryable vs IEnumerable

6. Delegates, Events, and Functional Constructs

- 6.1. Delegates (Func, Action, Predicate)
- 6.2. Multicast delegates
- 6.3. Events (event keyword, publisher/subscriber pattern)
- 6.4. Lambda expressions

7. Asynchronous Programming

- 7.1. Threads vs Tasks (conceptual)
- 7.2. Task and Task<T>
- 7.3. async/await keywords
- 7.4. Task composition (Task.WhenAll, Task.WhenAny)
- 7.5. Cancellation (CancellationToken)
- 7.6. Exception handling in async code
- 7.7. IAsyncEnumerable<T> and async streams
- 7.8. Common async pitfalls (ConfigureAwait, sync-over-async, deadlocks)
- 7.9. Task.Run vs async — CPU-bound work vs I/O-bound work

8. Memory Management (Lightweight)

- 8.1. Stack vs heap (concept only)
- 8.2. IDisposable and the using statement/declaration

9. SOLID Principles and Core Design Patterns

- 9.1. Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion
- 9.2. Repository pattern
- 9.3. Factory pattern
- 9.4. Strategy pattern
- 9.5. Manual Dependency Injection (concept, constructor injection)

10. Unit Testing and Debugging

- 10.1. Debugging tools (breakpoints, watch, call stack)
- 10.2. xUnit basics
- 10.3. Assertions, test lifecycle (Setup/Teardown)
- 10.4. Mocking (Moq)
- 10.5. Test-driven development concept
- 10.6. Integration testing basics

11. Version Control (Git)

- 11.1. Init, clone, commit, push, pull
- 11.2. Branching and merging
- 11.3. Resolving merge conflicts
- 11.4. Pull requests, commit conventions

12. Modern .NET — Application Hosting & Configuration

- 12.1. .NET CLI and project/solution structure (csproj, sln)
- 12.2. Dependency Injection container (IServiceCollection)
- 12.3. DI service lifetimes — Transient, Scoped, Singleton
- 12.4. Configuration system (appsettings.json, environment variables, user secrets, options pattern)
- 12.5. Logging (ILogger, providers)
- 12.6. Hosting model (Generic Host, WebApplicationBuilder)
- 12.7. Environments (Development/Staging/Production)

13. ASP.NET Core — Web API Fundamentals

- 13.1. Routing (attribute routing, conventional routing)
- 13.2. Controllers and actions
- 13.3. Minimal APIs
- 13.4. Model binding and validation
- 13.5. DTOs and mapping (manual or AutoMapper)
- 13.6. Middleware pipeline (built-in and custom)
- 13.7. Filters (action filters, exception filters)
- 13.8. Content negotiation, status codes, ProblemDetails
- 13.9. API versioning
- 13.10. Swagger/OpenAPI documentation
- 13.11. Health checks (IHealthCheck)
- 13.12. Rate limiting

14. Entity Framework Core

- 14.1. Connection strings and provider setup (SQL Server, PostgreSQL, SQLite)
- 14.2. DbContext and DbSet<T>
- 14.3. Code-first modeling, data annotations, Fluent API
- 14.4. Migrations
- 14.5. Relationships (one-to-one, one-to-many, many-to-many)
- 14.6. Querying with LINQ-to-Entities, Include/ThenInclude
- 14.7. Tracking vs no-tracking queries
- 14.8. Transactions
- 14.9. Concurrency handling (optimistic concurrency, RowVersion)
- 14.10. Performance considerations (N+1 problem, projections, pagination)

15. Authentication and Authorization

- 15.1. ASP.NET Core Identity
- 15.2. Claims-based identity
- 15.3. JWT-based authentication
- 15.4. Refresh tokens
- 15.5. Role-based and policy-based authorization

16. API Communication

- 16.1. HttpClient and IHttpClientFactory
- 16.2. CORS configuration

17. Clean Architecture / Project Structuring

- 17.1. Layered architecture (Domain, Application, Infrastructure, API)
- 17.2. Separation of concerns across layers
- 17.3. Repository and Unit of Work pattern in practice

TIER 2 — SHOULD KNOW

Understand the concept and recognize it in code. Go deep only when a specific project or job requirement needs it — don't pre-study these.

- Indexers and operator overloading (OOP extras)
- Covariance and contravariance (generics)
- Immutable collections, `Span<T>` and `Memory<T>` (performance-oriented collections)
- Anonymous methods, closures, expression trees (functional constructs depth)
- Multithreading depth — `Parallel.For/ForEach`, concurrent collections, `Semaphore/Mutex`/synchronization primitives
- Garbage collection internals, finalizers, boxing/unboxing depth
- Reflection and custom attributes
- File I/O and serialization — JSON serialization (learn now, used constantly); XML serialization and raw file streams (defer)
- Singleton, Decorator, Observer design patterns
- Caching — `IMemoryCache`, distributed caching (Redis concept-level)
- Real-time communication — SignalR fundamentals, WebSockets concept
- Background processing — `IHostedService/BackgroundService`, scheduled jobs (Hangfire/Quartz concept-level)
- Resilience patterns — retries, timeouts (Polly concept-level)
- CQRS concept and MediatR
- Raw SQL and stored procedures via EF Core
- Database providers comparison depth (beyond basic setup)
- Deployment basics — publish profiles, dotnet publish

TIER 3 — AWARE OF ONLY

No study time required. Just know these exist and roughly what they're for, in case they come up in conversation or a specific job requires them.

Blazor

A C#-based alternative to JavaScript frontends (Blazor Server, Blazor WebAssembly, Blazor Hybrid). Skipped because Angular is the chosen frontend.

ASP.NET Web Forms

Legacy page-based web framework with ViewState and server controls, from early .NET Framework era.

ASP.NET MVC 5

Pre-Core version of MVC, differs in conventions and setup from ASP.NET Core MVC.

Web API 2

Pre-Core version of Web API.

Entity Framework 6 (EF6)

Predecessor to EF Core, differs in APIs and configuration.

web.config

Legacy XML-based configuration system, replaced by appsettings.json in modern .NET.

WCF (Windows Communication Foundation)

SOAP-based service framework from .NET Framework era.

Global Assembly Cache (GAC)

Legacy machine-wide assembly storage and versioning system.

System.Web namespace

Legacy web hosting dependencies, replaced by Kestrel/cross-platform hosting.

IIS hosting specifics

Windows-specific hosting model, vs. Kestrel/cross-platform hosting in modern .NET.

WPF and WinForms

Legacy Windows-only desktop UI frameworks.

Note on ordering: Within Tier 1, sections are listed in dependency order — each section assumes everything before it has been learned. Section 0 (Foundations Before Code) exists because EF Core and ASP.NET Core both assume prior knowledge of HTTP, REST, and basic SQL/relational database concepts that are easy to take for granted but essential for a true zero-knowledge starting point.